

5. Production du code intermédiaire

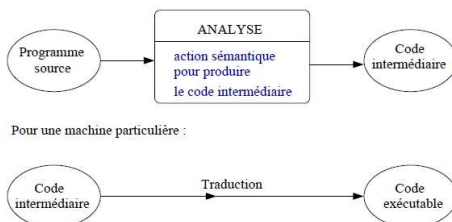
Chap5. Production du code intermédiaire

Plan

- I. Code intermédiaire
- II. Code trois adresses
- III. Code à base d'arbre
- IV. Niveau du langage
- V. Structures de contrôle
- VI. Optimisations
- VII. Réduction de force
- VIII. Déroulage de boucles

Code intermédiaire

On peut comparer le code intermédiaire à un assembleur universel (indépendant de la machine cible) :



Code intermédiaire (2)

2ème avantage : il est possible de prévoir un optimiseur de code intermédiaire qui est indépendant de la machine cible.

Les arbres abstraits (notation postfixée) sont déjà des représentations intermédiaires.

Nous souhaitons maintenant passer d'une représentation arborescente à une représentation linéaire.

Code intermédiaire (3)

- Le code intermédiaire est un code proche des assembleurs des machines cibles, mais suffisamment général pour convenir pour chacune d'elles
- Historiquement, le langage choisi est du code trois adresses : une instruction contient une opération, une ou deux opérandes, et une adresse pour stocker le résultat (sauf fonctions)
- On peut aussi conserver le stockage des expressions sous forme d'arbre, plus facile à compiler à l'aide d'une pile
- Dans les deux cas, on utilise un nombre illimité et non optimisé de variables, et des instructions de sauts

Code intermédiaire (4)

Code intermédiaire : format utilisé dans la compilation pour faciliter la transformation du code source dans un langage de haut niveau vers un langage de bas niveau, comme le langage machine. Il permet également d'effectuer des optimisations sur le code intermédiaire avant de le traduire en code machine.

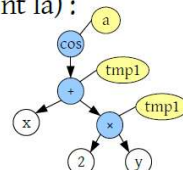
Code trois adresses : langage intermédiaire utilisé dans la compilation pour représenter les instructions du programme de manière simplifiée.

Code trois adresses

- Le code trois adresses nécessite de stocker explicitement les résultats intermédiaires des expressions dans une variable temporaire (créée à ce moment là) :

– $a = \cos(x + 2 * y)$ devient

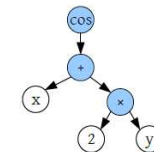
- $tmp1 = 2 * y$
- $tmp2 = x + tmp1$
- $a = \cos(tmp2)$



- Pour le générer à partir de l'AST, il suffit d'attribuer une nouvelle variable temporaire à chaque nœud intermédiaire de l'arbre.

Code à base d'arbre

- Les expressions étant stockées dans un arbre, il n'y a rien à faire de plus :
– $a = \cos(x + 2 * y)$ reste



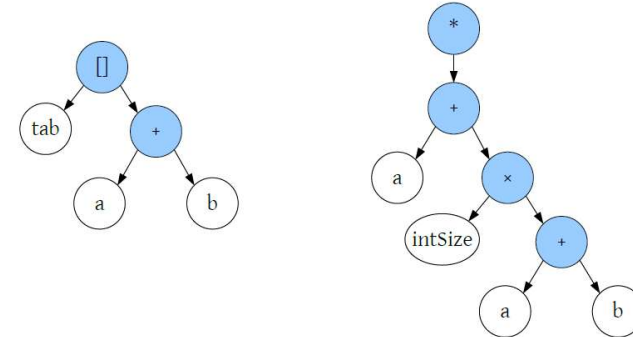
- Est commun aux deux styles codes : la compilation des structures de contrôles et le choix du niveau du langage intermédiaire.

Niveau du langage

- En fonction des architectures cibles, on peut choisir le niveau du langage intermédiaire
 - par exemple, on peut avoir des instructions spécifiques aux tableaux, ou directement les transformer en déréférencement de pointeurs
 - en choisissant de tout passer en arithmétique pointeurs, il peut être plus difficile de profiter des opérations natives offertes par les processeurs modernes (MMX, SSE2...) ou de compiler du java
 - à l'inverse, en remplaçant tout par des pointeurs, il est plus facile d'écrire le générateur de code

Exemple

$x = \text{tab}[a+b]$



Structures de contrôle

- Les structures de contrôles sont remplacées par du code basé sur des tests et des gotos
- Pour certaines comme le switch, il y a plusieurs implémentations possibles
- Le choix doit se faire à *compile time*, sans toujours savoir quelles seront les valeurs prises pendant l'exécution du programme
- Deux approches
 - profiling
 - machine virtuelle et JIT

Boucles

- for($i=0; i<12; i++$)
 - $i=0$
 - test_for :
 - if ($i \geq 12$) goto fin_for
 - optimisation : supprimer test
 - debut_for :
 - instructions du bloc
 - continue :
 - $i++$
 - goto test_for
 - fin_for : ...
 - $i=0$
 - if ($i \geq 12$) goto fin_for
 - optimisation : supprimer test
 - debut_for :
 - instructions du bloc
 - continue :
 - $i++$
 - if ($i < 12$) goto debut_for
 - fin_for : ...

Switch

- Deux manières principales
 - suite de if/else, si aucune autre ne fonctionne
 - tableau de labels : goto label[variable_du_switch-minimum], si les valeurs sont agglomérées autour d'une seule
- Deux autres plus exotiques
 - recherche dichotomique, si les cas sont ordonnés et nombreux
 - table de hachage, si on peut trouver une bonne fonction de hachage

Switch (2)

- if (a==10)
 - goto L10
- else if (a==11)
 - goto L11
- ...
- else if (a==18)
 - goto L18
- else
 - goto default
- Dans le segment des constantes, on a le tableau :
 - label[] = {L10,L11,L12,...}
 - (les Lx sont bien sûr remplacés par l'adresse de l'instruction)
- if (a<10 || a>18)
 - goto default
- else
 - goto label[a-10]

Switch (3)

- Quand les valeurs sont proches, mais avec des «trous», on peut mettre des defaults dans le tableau de label
- switch(a) {
 - case 3:...
 - case 2:...
 - case 6:...
 - default:...
 - case 5:...
 - case 8:...
- label[] = {L2,L3,default,L5,L6,default,L8}
- if (a<2 || a>8)
 - goto default
- else
 - goto label[a-2]

Optimisations

- Parmi les plus simples
 - précalcul des constantes
 - dans l'AST, ajouter un attribut qui dit un nœud est une constante
 - si tous les sous-attributs sont constants, le nœud devient constant
 - attention aux appels de fonctions dynamiques, le code peut changer entre la compilation et l'exécution
 - suppression du code inatteignable
 - réutilisation des expressions déjà calculées
 - attention aux multithread si les variables sont «volatiles»
 - si i est volatile, i+1 peut avoir changé de valeur si une autre thread a modifié i, et le spécification impose que le calcul reflète le changement.

Réduction de force

- Pour les boucles où le compteur est seulement utilisé multiplié par une constante (parcours de tableau)
- `for(i=0;i<10;i++) {
 val=(tab+4*i)
 println(val)
}`
- `for(i=tab;i<40+tab;i+=4) {
 val = *i
 println(val)
}`
- Dans gcc, l'option est `-fstrength-reduce` (active par défaut)

Déroulage de boucles

- On peut choisir de dérouler les boucles dont on connaît la taille :
 - `for(i=0;i<8;i++) instructions(i)`
devient
– `instructions(0) instructions(1) instructions(2) instructions(3)
instructions(4) instructions(5) instructions(6) instructions(7)`
 - `for(i=0;i<80;i++) instructions`
devient
– `for(i=0;i<80;i+=8) {
 instructions(i) instructions(i+1) instructions(i+2) instructions(i+3)
 instructions(i+4) instructions(i+5) instructions(i+6) instructions(i+7)
}`

Déroulage de boucles (2)

- On peut aussi dérouler les boucles dont on ne connaît pas la taille, en dupliquant aussi les tests
 - `for(i=0;i<n;i++) print(i)`
devient
– `for(;;) {
 • print(i); i++; if(i>=n) break;
 • print(i); i++; if(i>=n) break;
 • ...
 • print(i); i++; if(i>=n) break;
– }`

Déroulage de boucles (3)

- L'intérêt de déroulage de boucle est, dans le cas «taille connue» de limiter les tests, et aussi de limiter le nombre de sauts qui détruit le pipeline des processeurs modernes
- L'option pour dérouler les boucles de taille connue est
– `-funroll-loops`
- L'option pour dérouler toutes les boucles est
– `-funroll-all-loops`

PRODUCTION DE CODE INTERMEDIAIRE

Pour avoir le code à trois adresses, chaque instruction est divisée en trois parties :

1. Opérande gauche (L-value) : la destination de l'opération
2. Opérateur : l'opération à effectuer
3. Opérande droit (R-value) : la source de l'opération

Exemple de code à trois adresses :

$t1 = a + b$

$t2 = c * t1$

Où $t1$, $t2$, a , b et c sont des variables, et les opérations effectuées sont l'addition et la multiplication

PRODUCTION DE CODE INTERMEDIAIRE

- Exercice : Produire le code à trois adresses

```

PROGRAM max
BEGIN
  int a, b, c;

  read (a) ;
  read (b) ;

  IF a < b THEN
    print ("a est plus petit b");
  ELSE
    print ("a est plus grand b");
  END IF

  c := b * b + 4 * a * a ;

  print (c);
END
100 lire (a)
101 lire (b)
102 t1 = a < b
103 if t1 = 0 GOTO 106
104 write ("print (\a est plus petit que b\");")
105 GOTO 107
106 write ("print (\a est plus grand que b\");")
107 t3 = b * b
108 t4 = a * a
109 t5 = 4 * t4
110 t6 = t3 + t5
111 write ("print (c);")

```